

Maestría en  
Software

---

# Patrones de Diseño de Software

Autor(es):      Mauricio Ortiz Ochoa  
mortizo@ups.edu.ec

Tema:

---

# Unidad 01. Patrones de diseño creacionales

Tema:

---

# Diseño de clases

- *Diseño de clases*
- Código limpio

## Diseño de Clases

---

# La necesidad de diseñar software

- La complejidad aumenta con el número de reglas, componentes, dependencias y cambios.
- Los requisitos establecen qué debe lograr el sistema; el diseño define cómo se organizará la solución.
- Los modelos permiten analizar responsabilidades, relaciones y alternativas antes de comprometerse con el código.
- Un diseño compartido facilita la comunicación y coordinación entre los integrantes del equipo.
- El diseño no es un plano inmutable: se revisa y mejora conforme aprendemos sobre el problema y la solución.

Requisitos: qué → Diseño: cómo → Código: implementación

## Diseño de Clases

# No existe la bala de plata

Brooks, F. P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering.

- **Complejidad:** múltiples componentes, estados, reglas y dependencias interactúan entre sí.
- **Conformidad:** el software debe ajustarse a procesos, regulaciones, interfaces y sistemas existentes.
- **Cambio continuo:** los requisitos y el contexto evolucionan; el software exitoso recibe nuevas modificaciones.
- **Invisibilidad:** su estructura conceptual no puede observarse directamente; cada modelo representa solamente una perspectiva.

## ¿La IA generativa es la nueva bala de plata?

- Puede acelerar el modelado, la generación de código, las pruebas y la documentación.
- Sin embargo, no determina por sí sola el contexto, las restricciones ni las consecuencias de una decisión de diseño.

La IA reduce trabajo accidental, pero no elimina la complejidad esencial ni reemplaza el criterio profesional.

## Diseño de Clases

# UML

UML = Unified Modeling Language.

- Lenguaje visual estandarizado por el Object Management Group (OMG) para representar diferentes perspectivas de un sistema.
- Surgió de la unificación de los métodos de Grady Booch, Ivar Jacobson y James Rumbaugh.
- Proporciona una notación compartida, pero no determina una metodología ni un proceso de desarrollo.
- Un modelo selecciona los elementos necesarios para comunicar y analizar decisiones de diseño; no pretende reproducir todo el código.

## Diagrama de Clases

- Representa la estructura estática del sistema.
- Muestra clases, interfaces, atributos, operaciones y relaciones.
- Permite analizar responsabilidades y colaboraciones antes y durante la implementación.
- Debe evolucionar junto con el diseño y el código.



OMG® Unified Modeling Language® (OMG UML®)

Version 2.5.1

OMG Document Number: formal/2017-12-05

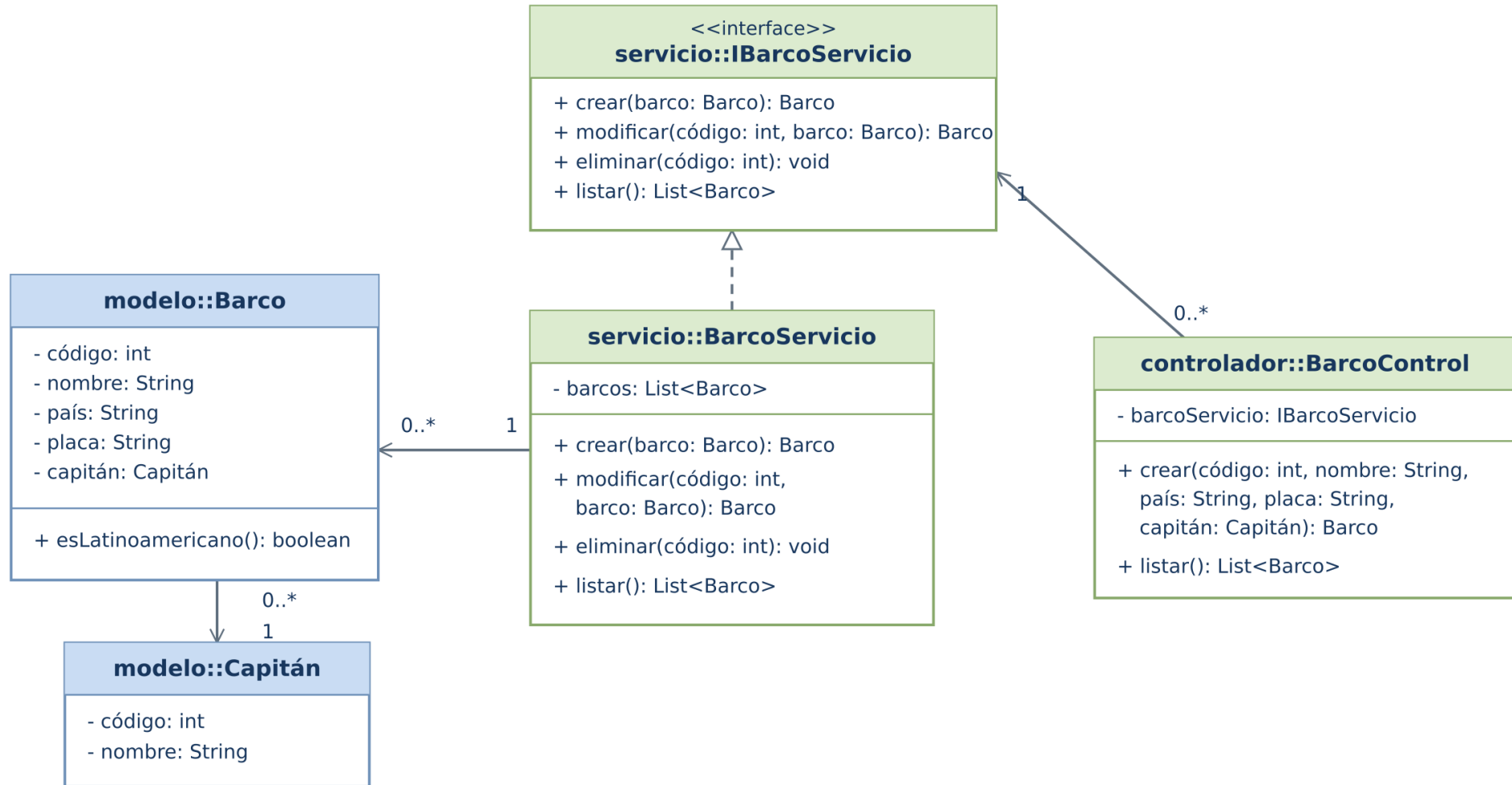
Date: December 2017

Normative URL: <https://www.omg.org/spec/UML/>

Machine Readable:

Normative: <https://www.omg.org/spec/UML/20161101/PrimitiveTypes.xmi>  
<https://www.omg.org/spec/UML/20161101/UML.xmi>  
<https://www.omg.org/spec/UML/20161101/StandardProfile.xmi>  
<https://www.omg.org/spec/UML/20161101/UMLDI.xmi>

## Diseño de Clases



## Diseño de Clases

# Constructores

- Un constructor establece el estado inicial de una nueva instancia.
- En Java, tiene el mismo nombre de la clase y no declara tipo de retorno.
- Se ejecuta al crear una instancia mediante new.
- Su declaración es opcional: si no se declara ninguno, Java proporciona un constructor sin parámetros.
- Cuando se declara al menos un constructor, Java deja de generar automáticamente el constructor por defecto.
- Un constructor debe asegurar que el objeto comience en un estado válido.

```
public class Barco {

    private String codigo;
    private String nombre;
    private Capitan capitan;

    public Barco(String codigo, String nombre, Capitan capitan) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.capitan = capitan;
    }
}
```

## Diseño de Clases

# Sobrecarga: misma intención, distintas entradas

- Consiste en declarar varios métodos o constructores con el mismo nombre y diferentes listas de parámetros.
- La diferencia puede estar en el número, tipo u orden de los parámetros.
- El tipo de retorno, por sí solo, no permite sobrecargar un método en Java.
- Es apropiada cuando todas las variantes representan la misma responsabilidad.
- Las implementaciones deberían delegar en una versión principal para evitar duplicación.
- Demasiadas variantes pueden volver ambigua la creación del objeto.

```
public class Barco {

    private String codigo;
    private String nombre;
    private String pais;

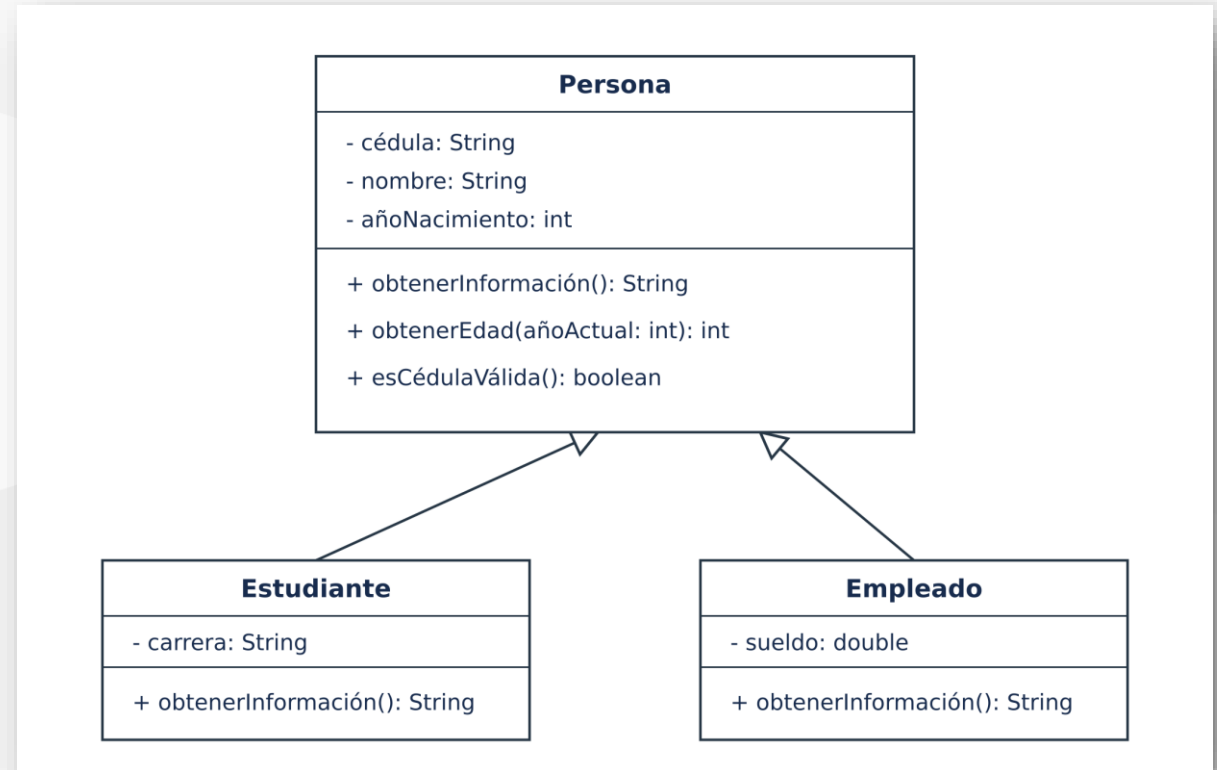
    public Barco(String codigo, String nombre) {
        this(codigo, nombre, "Ecuador");
    }

    public Barco(String codigo, String nombre, String pais) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.pais = pais;
    }
}
```

## Diseño de Clases

# Clases Abstractas: contrato y comportamiento común

- Representan conceptos generales que no deben instanciarse directamente.
- Pueden declarar atributos, constructores y métodos concretos.
- También pueden declarar métodos abstractos: especifican una operación, pero no proporcionan su implementación.
- Toda subclase concreta debe implementar los métodos abstractos heredados.
- Una subclase abstracta puede posponer esa implementación.
- Son apropiadas cuando las subclases comparten identidad, estado y comportamiento dentro de una relación “es un”.
- La herencia debe expresar una relación conceptual válida, no utilizarse únicamente para reutilizar código.



## Diseño de Clases

# Clases Abstractas

```
public abstract class Persona {

    private final String cedula;
    private final String nombre;
    private final int anioNacimiento;

    protected Persona(
        String cedula,
        String nombre,
        int anioNacimiento) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.anioNacimiento = anioNacimiento;
    }

    public abstract String obtenerInformacion();

    public int obtenerEdad(int anioActual) {
        return anioActual - anioNacimiento;
    }

    protected String getNombre() {
        return nombre;
    }
}
```

```
public final class Empleado extends Persona {

    private final double sueldo;

    public Empleado(
        String cedula,
        String nombre,
        int anioNacimiento,
        double sueldo) {
        super(cedula, nombre, anioNacimiento);
        this.sueldo = sueldo;
    }

    @Override
    public String obtenerInformacion() {
        return "El empleado se llama " + getNombre()
            + " y su sueldo es " + sueldo;
    }
}
```

## Diseño de Clases

# Sobrescritura de métodos (override)

- Una subclase puede proporcionar una implementación específica de un método heredado.
- El método sobrescrito mantiene la misma firma y un tipo de retorno compatible.
- La anotación `@Override` permite verificar que realmente se está sobrescribiendo un método.
- No se puede reducir la visibilidad definida por la clase base.
- `this` representa la instancia actual.
- `super.metodo()` permite reutilizar explícitamente la implementación de la clase base.
- Cuando se invoca el método mediante una referencia base, Java ejecuta la implementación correspondiente al tipo real del objeto.

```
public abstract class Persona {

    private final String nombre;

    protected Persona(String nombre) {
        this.nombre = nombre;
    }

    public String obtenerInformacion() {
        return "Nombre: " + nombre;
    }
}
```

```
public final class Empleado extends Persona {

    private final double sueldo;

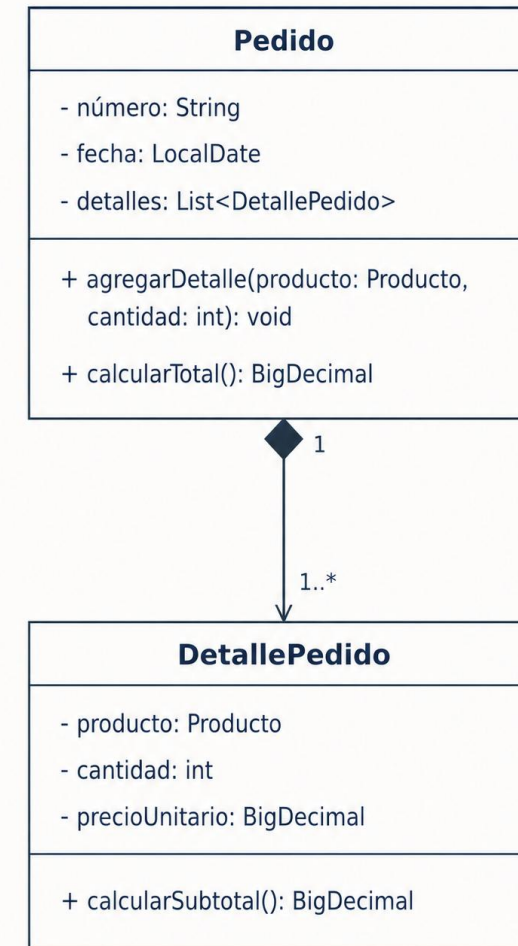
    public Empleado(String nombre, double sueldo) {
        super(nombre);
        this.sueldo = sueldo;
    }

    @Override
    public String obtenerInformacion() {
        return super.obtenerInformacion()
            + ", sueldo: " + this.sueldo;
    }
}
```

## Diseño de clases

# Composición: propiedad y ciclo de vida

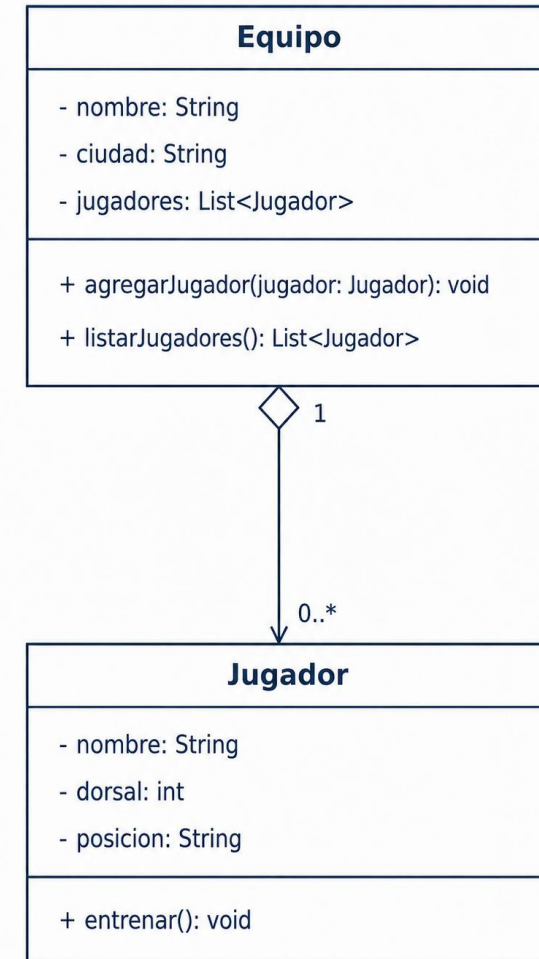
- Es una relación fuerte entre un todo y sus partes.
- El todo controla la pertenencia y el ciclo de vida de las partes dentro del modelo.
- Una parte pertenece como máximo a un único compuesto al mismo tiempo.
- Si el todo desaparece, sus partes dejan de existir como elementos independientes del modelo.
- Se representa mediante un rombo negro en el extremo del todo.
- Debe utilizarse únicamente cuando exista una dependencia real de ciclo de vida.



## Diseño de clases

# Agregación: relación todo–parte

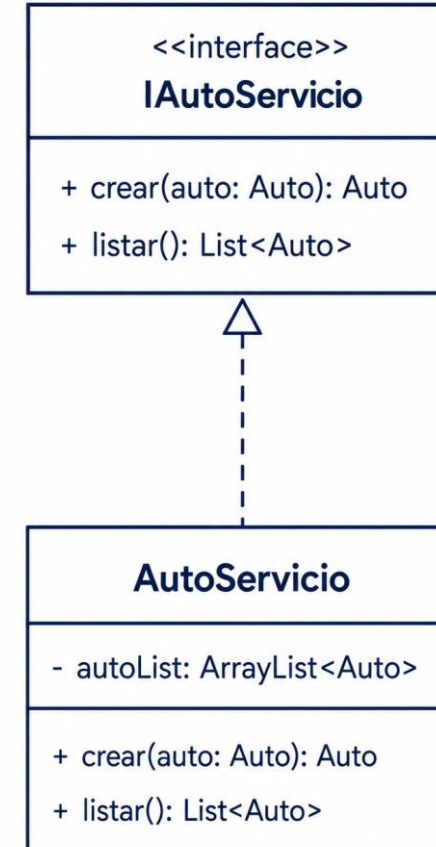
- Es una relación entre un todo y sus partes, pero más débil que la composición.
- El todo agrupa o utiliza las partes, pero no controla necesariamente su ciclo de vida.
- Una parte puede existir independientemente del todo.
- Si el todo desaparece, sus partes pueden continuar existiendo dentro del modelo.
- Se representa mediante un rombo blanco en el extremo del todo.
- Debe utilizarse cuando exista una relación conceptual “tiene un”, pero sin dependencia estricta de ciclo de vida.



## Diseño de Clases

# Interfaces

- Define un contrato de comportamiento que una clase puede implementar.
- Declara las operaciones que el objeto debe ofrecer, sin imponer cómo resolverlas.
- Una clase puede implementar una o varias interfaces.
- Permite programar contra abstracciones y reducir el acoplamiento.
- En Java puede contener métodos abstractos, default, static y private.
- Los campos declarados en una interfaz son implícitamente public static final.
- Una clase concreta debe implementar los métodos abstractos que correspondan.



## Actividad en clase: diagrama completo

# Caso: sistema de pedidos en línea

Una tienda necesita modelar su proceso de compra:

- Un cliente puede registrar varios pedidos.
- Cada pedido contiene uno o más detalles; cada detalle registra un producto, la cantidad y el precio aplicado.
- Los productos pueden ser físicos o digitales y calculan su costo de entrega de manera diferente.
- Un catálogo organiza los productos disponibles, aunque estos pueden existir independientemente del catálogo.
- Para confirmar un pedido, el sistema utiliza un servicio externo de pagos.
- Los atributos sensibles deben permanecer encapsulados.

### Reto del grupo

- Construir un diagrama UML que represente correctamente las clases, responsabilidades y relaciones del caso.

## Actividad en clase: diagrama completo

# Caso: sistema de pedidos en línea

El diagrama debe incluir:

- Clases con atributos y métodos relevantes.
- Visibilidad UML: +, – o #.
- Tipos de datos y valores de retorno.
- Multiplicidades en ambos extremos.
- Navegabilidad cuando sea necesaria.
- Al menos una asociación, una composición, una agregación, una generalización y una dependencia.
- Responsabilidades distribuidas coherentemente entre las clases.

Uso permitido de IA:

- Pueden utilizar IA como revisora, pero el grupo debe decidir y justificar el diseño final.
- Registrar el prompt utilizado y una recomendación de la IA que hayan aceptado, modificado o rechazado.

Entrega:

- Diagrama UML exportado en PDF o PNG y explicación oral de tres decisiones de diseño

Tema:

---

# Código limpio

- Diseño de clases
- *Código limpio*

## Código limpio

# ¿Por qué escribir un código limpio?

- Que el código funcione es indispensable, pero no garantiza que sea comprensible ni fácil de modificar.
- Sin disciplina de diseño, el crecimiento del sistema incrementa el costo y el riesgo de cada cambio.
- El código se lee y modifica muchas más veces de las que se escribe.
- La duplicación, la complejidad y las responsabilidades confusas acumulan deuda técnica.
- Refactorizar mejora la estructura interna sin cambiar el comportamiento observable.

### Regla del Boy Scout

- Deja el código un poco más limpio de lo que lo encontraste.



Código limpio

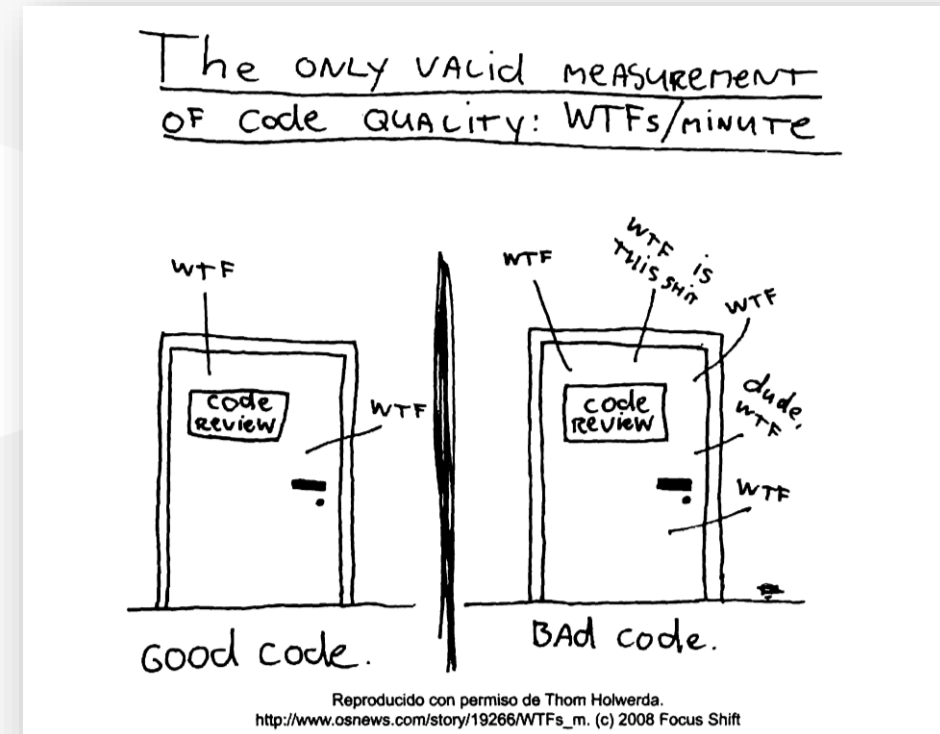
## Deuda Técnica

|             | Imprudente                                                                                                                                                                                                       | Prudente                                                                                                                                                                                                   |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deliberada  | <ul style="list-style-type: none"> <li>No hay tiempo para diseñar.</li> <li>Copiar y pegar sin evaluar consecuencias.</li> <li>Omitir pruebas o refactorización sin un plan.</li> </ul>                          | <ul style="list-style-type: none"> <li>Se acepta temporalmente por una restricción concreta.</li> <li>Se registran sus impactos y riesgos.</li> <li>Se define cuándo y cómo reducirla.</li> </ul>          |
| Inadvertida | <ul style="list-style-type: none"> <li>Desconocimiento de principios de diseño.</li> <li>Responsabilidades y dependencias mal distribuidas.</li> <li>La complejidad aparece durante el mantenimiento.</li> </ul> | <ul style="list-style-type: none"> <li>La implementación genera nuevo aprendizaje.</li> <li>Después descubrimos una solución mejor.</li> <li>Nuevos requisitos revelan limitaciones del diseño.</li> </ul> |

## Código limpio

# Dimensiones del código limpio

- Nombres significativos.
- Funciones pequeñas y cohesionadas.
- Comentarios necesarios y precisos.
- Formato consistente.
- Objetos y estructuras de datos.
- Manejo de errores.
- Límites e integraciones externas.
- Pruebas automatizadas.
- Clases con responsabilidades claras.



## Código limpio

# Nombres significativos

- Deben revelar la intención y utilizar el vocabulario del dominio.
- Deben ser pronunciables, buscables y suficientemente específicos.
- Utilizar el mismo término para el mismo concepto.
- Evitar abreviaturas ambiguas, nombres genéricos y codificaciones de tipo.
- Las clases se nombran con sustantivos; los métodos, con verbos.
- Los booleanos deben formularse como preguntas: isActive, hasPermission, canRetry.
- Utilizar un solo idioma de manera consistente según las convenciones del proyecto.

|             |                         |
|-------------|-------------------------|
| d           | elapsedDays             |
| lst         | activeCustomers         |
| flag        | isPaymentApproved       |
| tmp         | pendingOrder            |
| data        | invoiceDate             |
| process()   | calculateOrderTotal()   |
| doStuff()   | sendConfirmationEmail() |
| getIt()     | findCustomerById()      |
| usrMgr      | userManager             |
| productList | availableProducts       |

## Código limpio

# Funciones

- Pequeñas, enfocadas y con un solo propósito.
- Mantienen un nivel de abstracción coherente.
- Su nombre expresa claramente la acción realizada.
- Prefieren pocos parámetros; más de tres sugieren revisar el diseño.
- Evitan parámetros booleanos y efectos secundarios ocultos.
- Utilizan cláusulas de guarda para reducir el anidamiento.
- Extraen y reutilizan la lógica duplicada.
- Separan consultas de operaciones que modifican el estado.

|                                       |                                               |
|---------------------------------------|-----------------------------------------------|
| <code>process()</code>                | <code>calculateOrderTotal()</code>            |
| <code>doEverything()</code>           | <code>validateOrder()</code>                  |
| <code>save(a,b,c,d)</code>            | <code>saveOrder(order)</code>                 |
| <code>sendEmail(user,true)</code>     | <code>sendWelcomeEmail(user)</code>           |
| <code>update(order,status)</code>     | <code>markOrderAsPaid(order)</code>           |
| <code>validateAndSave()</code>        | <code>validateOrder()<br/>saveOrder()</code>  |
| <code>if(valid){if(paid){...}}</code> | <code>if (!valid) return;</code>              |
| <code>print(order,true)</code>        | <code>printDetailedOrder(order)</code>        |
| <code>duplicateDiscountCode()</code>  | <code>applyDiscount(order)</code>             |
| <code>getTotalAndSave()</code>        | <code>calculateTotal()<br/>saveOrder()</code> |

## Código limpio

# Principio DRY

- Don't repeat Yourself.
- Una regla o conocimiento del sistema debe tener una única representación autorizada.
- Evita duplicar reglas de negocio, validaciones, cálculos y transformaciones.
- Centraliza la lógica verdaderamente común en una función, clase o módulo.
- Reduce el riesgo de corregir una copia y olvidar las demás.
- Facilita que los cambios se realicen y prueben en un solo lugar.
- Refactoriza cuando las repeticiones representan el mismo concepto y tienen la misma razón para cambiar.



## Código limpio

# Comentarios

- No se debe documentar mediante comentarios aquello que el propio código puede expresar.
- Los nombres, funciones, clases y constantes deben revelar claramente la intención.
- Antes de añadir un comentario, se debe intentar mejorar o refactorizar el código.
- Los comentarios desactualizados generan desinformación y deuda técnica.
- Comentar únicamente cuando sea necesario explicar un porqué no evidente: una restricción externa, una decisión de negocio, un riesgo o una consecuencia.

### 1. Nombre significativo

#### × EVITAR

```
// Días transcurridos
int d = calculateDays();
```

#### ✓ PREFERIR

```
int elapsedDays = calculateDays();
```

El nombre elimina el comentario.

### 2. Método expresivo

#### × EVITAR

```
// Calcula el total
process();
```

#### ✓ PREFERIR

```
calculateOrderTotal();
```

La intención está en el método.

### 3. Constante con significado

#### × EVITAR

```
// Máximo tres intentos
if (retryCount >= 3) stop();
```

#### ✓ PREFERIR

```
MAX_LOGIN_ATTEMPTS = 3;
if (retryCount >= MAX_LOGIN_ATTEMPTS)
    stopAuthentication();
```

La constante documenta la regla.

### 4. Excepción válida

#### ✓ COMENTAR EL PORQUÉ

```
// El proveedor bloquea la cuenta
// después de tres intentos.

if (retryCount >= MAX_LOGIN_ATTEMPTS) {
    stopAuthentication();
}
```

El código no revela esta restricción.

## Código limpio

# Formato

- El formato debe facilitar la lectura y hacer visible la estructura del código.
- Todo el proyecto debe aplicar las mismas reglas de sangría, espacios, llaves, saltos de línea e importaciones.
- Utilizar un formateador y un analizador estático configurados para todo el equipo.
- Integrar estas verificaciones en el IDE y en el proceso de integración continua.
- Agrupar instrucciones relacionadas y separar conceptos diferentes mediante espacios en blanco.
- Organizar los archivos desde el contrato público y los elementos generales hacia los detalles de implementación.
- Evitar alineaciones manuales que se rompen cuando cambia el código.
- Las convenciones del proyecto prevalecen sobre las preferencias personales.

### × FORMATO DEFICIENTE

```
public BigDecimal total(Order o){
    BigDecimal t=BigDecimal.ZERO;
    for(Item i:o.getItems()){
        if(i.isActive()){
            t=t.add(i.getPrice().multiply(
                BigDecimal.valueOf(i.getQty())));}}
    return t;}
```

### ✓ FORMATO CONSISTENTE

```
public BigDecimal total(Order o) {
    BigDecimal t = BigDecimal.ZERO;

    for (Item i : o.getItems()) {
        if (i.isActive()) {
            t = t.add(
                i.getPrice().multiply(
                    BigDecimal.valueOf(i.getQty())
                )
            );
        }
    }
    return t;
}
```

## Código limpio

# Objetos y estructuras de datos

- Una estructura de datos representa valores y ofrece poco comportamiento.
- Elegir conscientemente según el tipo de cambio esperado.
- Evitar híbridos que exponen datos y, al mismo tiempo, concentran comportamiento.
- No devolver colecciones internas modificables ni generar getters y setters indiscriminadamente.
- Aplicar la Ley de Demeter: colaborar únicamente con objetos conocidos directamente.

### × OBJETO QUE EXPONE SU ESTADO

```
class BankAccount {
    public BigDecimal balance;
}

account.balance =
    account.balance.subtract(amount);
```

Cualquier cliente puede romper las reglas del objeto.

### ✓ OBJETO QUE PROTEGE SUS INVARIANTES

```
class BankAccount {
    private BigDecimal balance;

    public void withdraw(BigDecimal amount) {
        if (amount.compareTo(balance) > 0) {
            throw new InsufficientFundsException();
        }
        balance = balance.subtract(amount);
    }
}
```

El comportamiento controla los cambios de estado.

## Código limpio

# Procesar Errores

- Utilizar excepciones para condiciones excepcionales, no como flujo normal de control.
- Preferir excepciones específicas del dominio.
- Capturar una excepción únicamente cuando pueda recuperarse, tratarse o enriquecerse.
- Conservar la causa original y añadir contexto útil.
- Evitar bloques catch vacíos y capturas genéricas.
- Representar explícitamente la ausencia mediante Optional, resultados tipados o colecciones vacías cuando corresponda.
- No incluir contraseñas, tokens ni datos sensibles en mensajes o registros.

### × OCULTA EL PROBLEMA

```
try {
    payment.charge(order);
} catch (Exception e) {
    return null;
}
```

Pierde la causa y obliga a interpretar un null.

### ✓ AÑADE CONTEXTO Y PRESERVA LA CAUSA

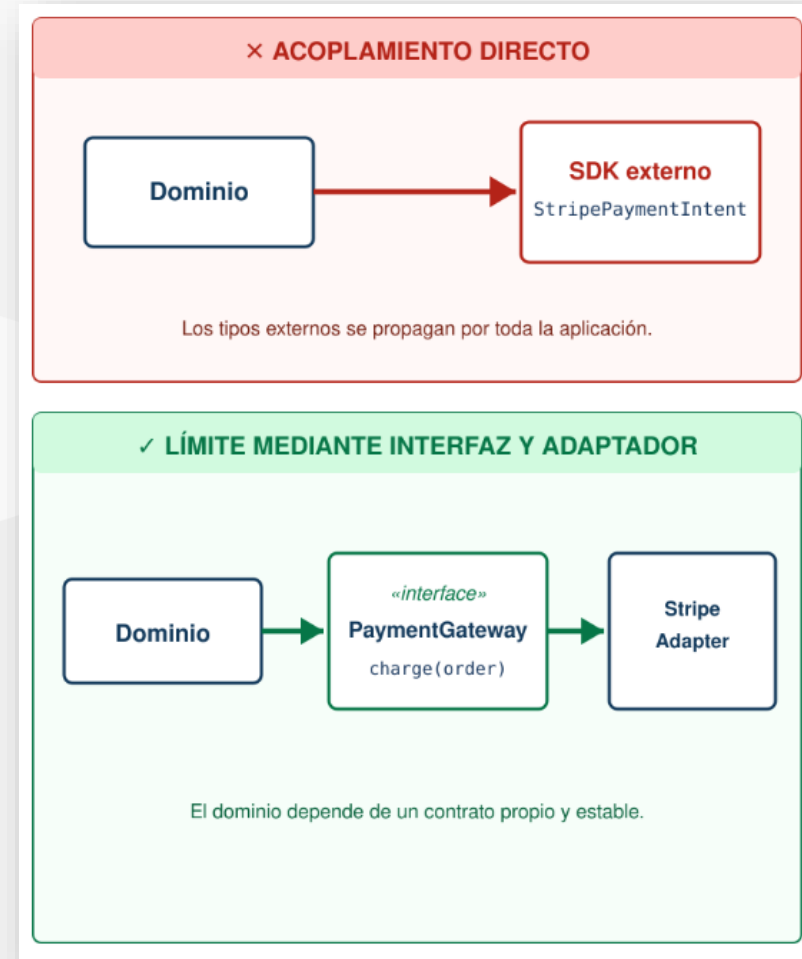
```
try {
    payment.charge(order);
} catch (GatewayException cause) {
    throw new PaymentFailedException(
        "No fue posible procesar el pedido "
        + order.getId(),
        cause
    );
}
```

El error conserva contexto sin exponer datos sensibles.

## Código limpio

# Límites

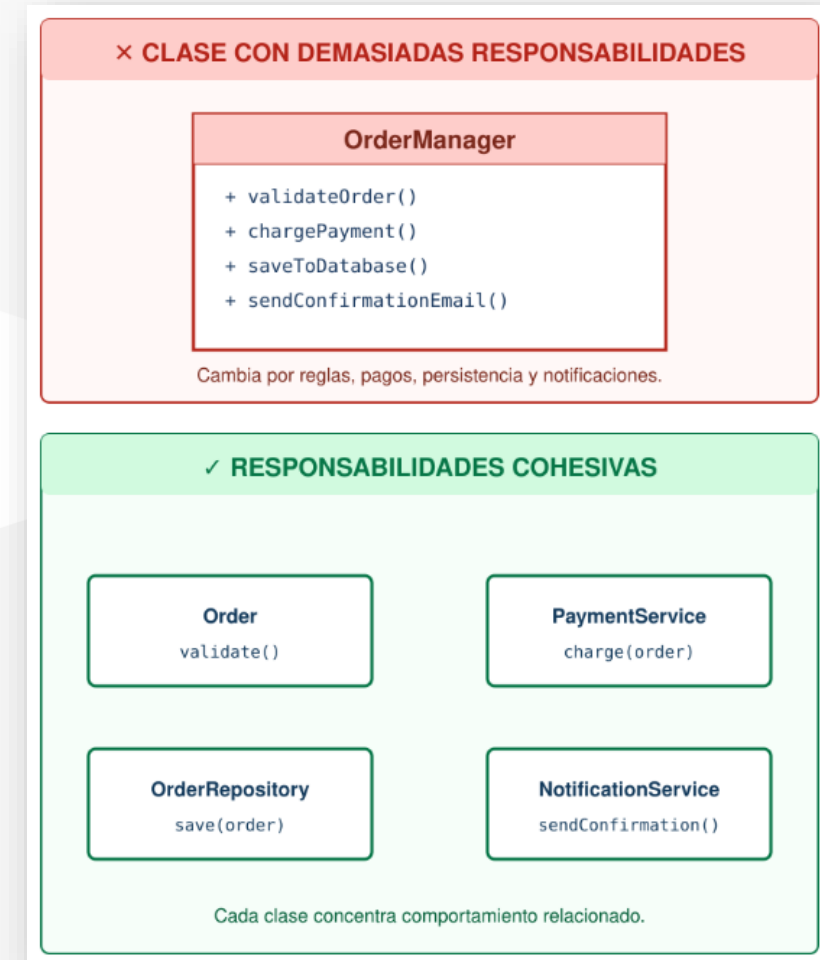
- Separar el dominio del código desarrollado por otros equipos o proveedores.
- Definir interfaces propias y estables para las necesidades del sistema.
- Encapsular bibliotecas externas mediante adaptadores.
- Evitar que tipos, excepciones y configuraciones de terceros se propaguen por toda la aplicación.
- Crear pruebas de aprendizaje o de contrato para verificar el comportamiento externo.
- Mantener versiones controladas y evaluar los cambios antes de actualizar dependencias.
- Documentar los supuestos y restricciones existentes en el límite.



## Código limpio

# Clases

- Una clase debe representar un concepto claro y tener una responsabilidad principal.
- Debe existir una razón coherente para que cambie.
- La cohesión es alta cuando sus métodos colaboran sobre un estado y propósito relacionados.
- Mantener una interfaz pública pequeña y ocultar los detalles de implementación.
- Distribuir persistencia, pagos, notificaciones y reglas de negocio en responsabilidades diferentes.
- Organizar sus elementos siguiendo una convención compartida por el equipo.
- Evaluar el tamaño por responsabilidades, no por un número arbitrario de líneas.



## Código limpio

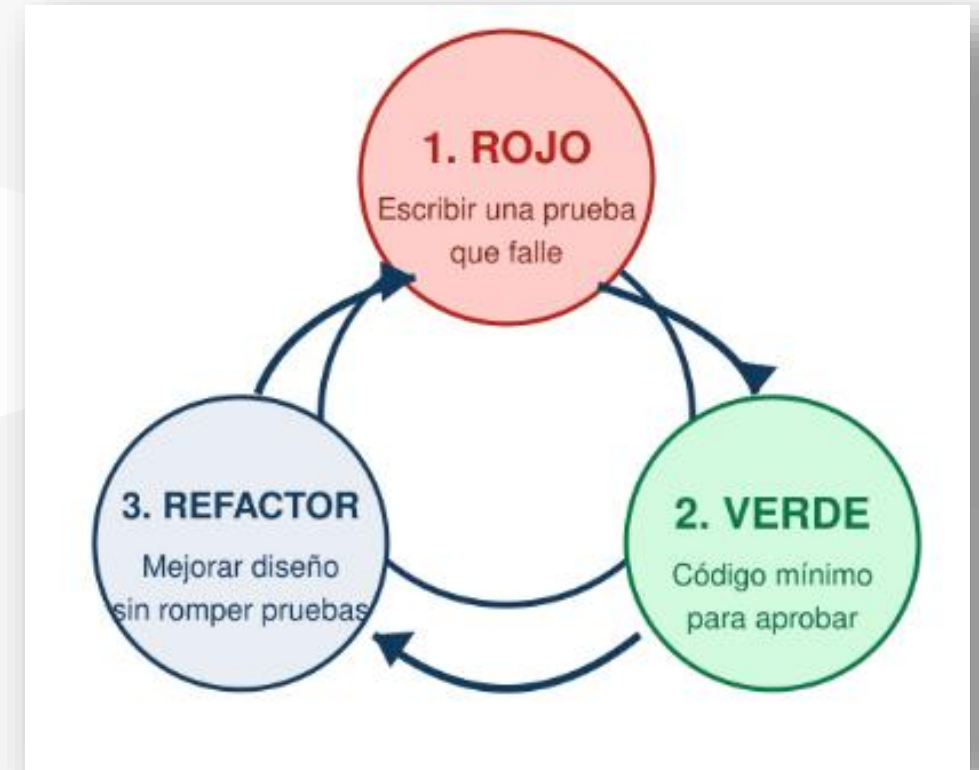
# Pruebas

### Ciclo TDD

- Rojo: escribir una prueba que describa el comportamiento esperado y comprobar que falla.
- Verde: implementar el código mínimo necesario para aprobarla.
- Refactorizar: mejorar el diseño manteniendo todas las pruebas aprobadas.
- Repetir el ciclo mediante incrementos pequeños.

### Las pruebas deben ser FIRST

- Fast: rápidas.
- Independent: independientes.
- Repeatable: repetibles.
- Self-validating: con resultado inequívoco.
- Timely: escritas oportunamente.



## Unidad 01.02

# Retroalimentación y cierre

### Qué vimos hoy

- El diseño de clases transforma los requisitos en responsabilidades, relaciones y decisiones implementables.
- UML permite representar clases, atributos, operaciones, multiplicidades, navegabilidad y dependencias.
- Los constructores deben garantizar que los objetos comiencen en un estado válido.
- Las clases abstractas y las interfaces permiten trabajar con abstracciones y reducir el acoplamiento.
- La composición controla el ciclo de vida de sus partes; la agregación permite que estas existan independientemente.
- La actividad grupal integró las relaciones UML y utilizó IA para revisar, no sustituir, las decisiones del equipo.
- El código limpio facilita comprender, modificar y probar el sistema.
- Los nombres, funciones, clases, límites, errores y pruebas influyen directamente en la mantenibilidad.

**Antes de finalizar:** ¿Qué decisión del diagrama o del código modificarían ahora y por qué?

**Idea final:** Un sistema mantenible no surge únicamente de código que funciona, sino de responsabilidades claras, dependencias controladas y decisiones que pueden comprenderse y justificarse.

## Anexos

# Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.

Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.

Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.

Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.

Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.

IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.

Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

## Anexos

---

## Links de Referencia

- <https://www.campusmvp.es/recursos/post/los-conceptos-fundamentales-sobre-programacion-orientada-objetos-explicados-de-manera-simple.aspx>
- <https://codideep.com/blogpost/stupid-vs-solid>
- <https://refactoring.guru/design-patterns>
- <https://reactiveprogramming.io/books/design-patterns/es>

Gracias por su atención

---

[posgrados.ups.edu.ec](http://posgrados.ups.edu.ec)